

Amazon SDE1 Coding Interview Guide

27 problems from your playlist. One page each. Brute force code AND optimal code, side by side.

A note on video transcripts: this session's network policy blocks youtube.com, so no transcripts could be pulled from the linked playlist. Every explanation below is written from general algorithmic knowledge, not transcribed from any video — see the per-problem note under each title. To get real transcripts pulled in, enable broader network access (or allowlist youtube.com) in this environment's settings and re-run.

Table of Contents

#	Problem	LC	Pattern
1	Two Sum	#1	Hash Map
2	Contains Duplicate	#217	Hash Set
3	Group Anagrams	#49	Hash Map
4	Product of Array Except Self	#238	Prefix/Suffix Products
5	Subarray Sum Equals K	#560	Prefix Sum + Hash Map
6	Valid Palindrome	#125	Two Pointers
7	Container With Most Water	#11	Two Pointers
8	Longest Substring Without Repeating Characters	#3	Sliding Window
9	Minimum Window Substring	#76	Sliding Window
10	Valid Parentheses	#20	Stack
11	Design Min Stack	#155	Stack (Auxiliary Min Stack)
12	Daily Temperatures	#739	Monotonic Stack
13	Evaluate Reverse Polish Notation	#150	Stack
14	Implement Queue using Stacks	#232	Stack (Two-Stack Trick)
15	Implement Stack using Queues	#225	Queue (Rotation Trick)
16	Maximum Depth of Binary Tree	#104	DFS / Recursion
17	Binary Tree Level Order Traversal	#102	BFS
18	Binary Tree Vertical Order Traversal	#314	BFS + Hash Map (Column Index)
19	Clone Graph	#133	DFS/BFS + Hash Map
20	Course Schedule	#207	Graph DFS / Topological Sort (Cycle Detection)
21	Binary Search	#704	Binary Search
22	Find Minimum in Rotated Sorted Array	#153	Binary Search (Modified)
23	LRU Cache	#146	Hash Map + Doubly Linked List
24	Random Pick with Weight	#528	Prefix Sum + Binary Search
25	Rotting Oranges	#994	Multi-Source BFS
26	Maximum Subarray	#53	Dynamic Programming (Kadane's Algorithm)
27	Top K Frequent Elements	#347	Bucket Sort (or Heap)

Two Sum

LeetCode #1 | **EASY** | Pattern: Hash Map

Video: "Two Sum - Leetcode 1 - HashMap - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an array of integers and a target, return the indices of the two numbers that add up to the target. Exactly one solution exists; can't use the same element twice.

Recognize it: "find two numbers that add up to X" + need better than $O(n^2)$ → Hash Map (store complements as you scan).

BRUTE FORCE

Check every pair of indices and test if they sum to target.

```
def two_sum(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

Complexity: $O(n^2)$ time | $O(1)$ space

OPTIMAL

Store each value's index as you scan; before adding, check if target minus current value was already seen — one pass instead of two nested loops.

```
def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: Duplicate values, e.g. `nums=[3,3]`, `target=6` — works because we check the map BEFORE inserting the current number, so a number never pairs with itself at the same index.

Memory hook: "I need the complement — remember what I've already seen."

Contains Duplicate

LeetCode #217 | **EASY** | Pattern: Hash Set

Video: "Contains Duplicate - Leetcode 217 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an array of integers, return true if any value appears at least twice, and false if every element is distinct.

Recognize it: "has any value appeared before" → Hash Set for $O(1)$ membership checks.

BRUTE FORCE

Compare every pair of elements for equality.

```
def contains_duplicate(nums):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j]:
                return True
    return False
```

Complexity: $O(n^2)$ time | $O(1)$ space

OPTIMAL

A hash set lets me check 'have I seen this before' in $O(1)$ average time, so one pass suffices.

```
def contains_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False

# One-liner alt: len(set(nums)) != len(nums)
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: Empty array or single-element array — loop never finds a duplicate, correctly returns False.

Memory hook: "Seen it before? Set tells you instantly."

Group Anagrams

LeetCode #49 | **MEDIUM** | Pattern: Hash Map

Video: "Group Anagrams - Categorize Strings by Count - Leetcode 49" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an array of strings, group the anagrams together (any order). Two strings are anagrams if they contain the same letters with the same counts.

Recognize it: "group strings that are permutations of each other" → canonical key + Hash Map.

BRUTE FORCE

Compare every string against every other by sorting both and checking equality.

```
def group_anagrams(strs):
    groups = []
    used = [False] * len(strs)
    for i in range(len(strs)):
        if used[i]:
            continue
        group = [strs[i]]
        for j in range(i + 1, len(strs)):
            if not used[j] and sorted(strs[i]) == sorted(strs[j]):
                group.append(strs[j])
                used[j] = True
        groups.append(group)
    return groups
```

Complexity: $O(n^2 * k \log k)$ time | $O(n * k)$ space

Edge case: Empty string in the list — its count key is all zeros, forms its own group.

Memory hook: "Same letter counts = same key = same bucket."

OPTIMAL

A 26-count tuple (or sorted string) is identical for all anagrams of a word, so it works as a hash map key that groups them in one pass.

```
from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)
    for word in strs:
        count = [0] * 26
        for ch in word:
            count[ord(ch) - ord('a')] += 1
        groups[tuple(count)].append(word)
    return list(groups.values())
```

Complexity: $O(n * k)$ time | $O(n * k)$ space

Product of Array Except Self

LeetCode #238 | **MEDIUM** | Pattern: Prefix/Suffix Products

Video: "Product of Array Except Self - Leetcode 238 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an array, return an array where each element is the product of all other elements, without using division, in $O(n)$.

Recognize it: "product/sum of everything except index i", no division allowed → Prefix * Suffix products.

BRUTE FORCE

For each index, loop over the rest of the array and multiply everything else.

```
def product_except_self(nums):
    n = len(nums)
    result = []
    for i in range(n):
        product = 1
        for j in range(n):
            if j != i:
                product *= nums[j]
        result.append(product)
    return result
```

Complexity: $O(n^2)$ time | $O(1)$ extra space

OPTIMAL

Answer at i = (product of everything left of i) * (product of everything right of i). Build prefix products left-to-right, then fold in suffix products right-to-left.

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]
    return result
```

Complexity: $O(n)$ time | $O(1)$ extra space

Edge case: Array with one zero — every index except the zero's own becomes 0; array with two zeros — every index becomes 0.

Memory hook: "Everything to my left, times everything to my right."

Subarray Sum Equals K

LeetCode #560 | **MEDIUM** | Pattern: Prefix Sum + Hash Map

Video: "Subarray Sum Equals K - Prefix Sums - Leetcode 560 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an array of integers and an integer k, return the total number of contiguous subarrays whose sum equals k. Numbers can be negative.

Recognize it: "number of subarrays summing to k", negatives allowed (rules out sliding window) → Prefix Sum + Hash Map.

BRUTE FORCE

Check every subarray's sum directly (or extend a running sum per start index).

```
def subarray_sum(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == k:
                count += 1
    return count
```

Complexity: $O(n^2)$ time | $O(1)$ space

OPTIMAL

$\text{sum}(i..j) = \text{prefix}[j] - \text{prefix}[i-1]$. So $\text{sum}(i..j) = k$ means $\text{prefix}[i-1] = \text{prefix}[j] - k$. Track how many times each prefix sum has occurred so far.

```
from collections import defaultdict

def subarray_sum(nums, k):
    count = 0
    prefix_sum = 0
    seen = defaultdict(int)
    seen[0] = 1 # empty prefix
    for num in nums:
        prefix_sum += num
        count += seen[prefix_sum - k]
        seen[prefix_sum] += 1
    return count
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: $\text{seen}[0]=1$ is required upfront — handles subarrays starting at index 0 whose sum is exactly k.

Memory hook: "Two equal prefix sums — the gap between them sums to k."

Valid Palindrome

LeetCode #125 | **EASY** | Pattern: Two Pointers

Video: "Valid Palindrome - Leetcode 125 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a string, return true if it's a palindrome after converting to lowercase and removing all non-alphanumeric characters.

Recognize it: "is this a palindrome", need $O(1)$ space → Two Pointers converging from both ends.

BRUTE FORCE

Build a cleaned version of the string, then compare it to its reverse.

```
def is_palindrome(s):
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]
```

Complexity: $O(n)$ time | $O(n)$ space

OPTIMAL

Use two pointers from both ends, skipping non-alphanumeric characters in place, comparing directly — no extra string built.

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Complexity: $O(n)$ time | $O(1)$ space

Edge case: String with only non-alphanumeric characters, e.g. ".,," — pointers cross without ever comparing, correctly returns True (empty string is a palindrome).

Memory hook: "Skip junk, compare from both ends inward."

Container With Most Water

LeetCode #11 | **MEDIUM** | Pattern: Two Pointers

Video: "Container with Most Water - Leetcode 11 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given heights of vertical lines, find two lines that together with the x-axis form a container holding the most water. Return the max area.

Recognize it: "maximize area between two lines/indices" → Two Pointers, always move the shorter side.

BRUTE FORCE

Check every pair of lines and compute the area they'd contain.

```
def max_area(height):
    best = 0
    n = len(height)
    for i in range(n):
        for j in range(i + 1, n):
            area = (j - i) * min(height[i], height[j])
            best = max(best, area)
    return best
```

Complexity: $O(n^2)$ time | $O(1)$ space

OPTIMAL

Start with the widest container (both ends). The shorter wall is always the bottleneck, so move it inward — moving the taller wall could never help.

```
def max_area(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        h = min(height[left], height[right])
        best = max(best, (right - left) * h)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

Complexity: $O(n)$ time | $O(1)$ space

Edge case: All lines the same height — area is maximized by the widest pair, which the pointers reach immediately at the start.

Memory hook: "Move the shorter wall — the taller one was never the limit."

Longest Substring Without Repeating Characters

LeetCode #3 | **MEDIUM** | Pattern: Sliding Window

Video: "Longest Substring Without Repeating Characters - Leetcode 3 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a string, find the length of the longest substring without repeating characters.

Recognize it: "longest substring with no repeats / unique characters" → Sliding Window + Hash Set.

BRUTE FORCE

Check every substring for uniqueness directly.

```
def length_of_longest_substring(s):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            if s[j] in seen:
                break
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best
```

Complexity: $O(n^2)$ time | $O(\min(n, \text{charset}))$ space

OPTIMAL

Expand the window right; if the new character is already in the window, shrink from the left until the duplicate is gone — window never revisits a position twice.

```
def length_of_longest_substring(s):
    seen = set()
    left = 0
    best = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
        seen.add(s[right])
        best = max(best, right - left + 1)
    return best
```

Complexity: $O(n)$ time | $O(\min(n, \text{charset}))$ space

Edge case: Empty string — loop never runs, best stays 0. All identical characters, e.g. "aaaa" — window size stays 1.

Memory hook: "Duplicate found? Shrink from the left until it's gone."

Minimum Window Substring

LeetCode #76 | **HARD** | Pattern: Sliding Window

Video: "Minimum Window Substring - Airbnb Interview Question - Leetcode 76" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given strings *s* and *t*, find the minimum window substring of *s* that contains every character of *t* (with at least its multiplicity), or empty string if none exists.

Recognize it: "smallest window containing all characters of another string" → Sliding Window + character-count matching.

BRUTE FORCE

Check every substring of *s* and test whether it contains all of *t*'s characters.

```
from collections import Counter

def min_window(s, t):
    need = Counter(t)
    best = ""
    for i in range(len(s)):
        for j in range(i, len(s)):
            window = Counter(s[i:j+1])
            if all(window[c] >= need[c] for c in need):
                if best == "" or j - i + 1 < len(best):
                    best = s[i:j+1]
    return best
```

Complexity: $O(n^3)$ time | $O(n)$ space

OPTIMAL

Expand right until the window satisfies all of *t*'s counts; then shrink from the left as much as possible while still valid, recording the smallest valid window found.

```
from collections import Counter

def min_window(s, t):
    if not t:
        return ""
    need = Counter(t)
    missing = len(t)
    left = 0
    best = (float('inf'), 0, 0)
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1
        while missing == 0:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2]+1]
```

Complexity: $O(n + m)$ time | $O(\text{charset})$ space

Edge case: *t* longer than *s*, or *t* has characters not in *s* — missing never reaches 0, correctly returns empty string.

Memory hook: "Expand until valid, then shrink until it breaks — record the smallest valid window."

Valid Parentheses

LeetCode #20 | **EASY** | Pattern: Stack

Video: "Valid Parentheses - Stack - Leetcode 20 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a string of just '()[]{}' characters, determine if the brackets are validly matched and nested.

Recognize it: "matching brackets / valid nesting" → Stack.

BRUTE FORCE

Repeatedly remove any adjacent matched pair like '()' , '[]', '{}' from the string until no more removals are possible; valid iff the string becomes empty.

```
def is_valid(s):
    pairs = ["()", "[]", "{}"]
    changed = True
    while changed:
        changed = False
        for p in pairs:
            if p in s:
                s = s.replace(p, "", 1)
                changed = True
    return len(s) == 0
```

Complexity: $O(n^2)$ time | $O(n)$ space

OPTIMAL

Push opening brackets. On a closing bracket, it must match the top of the stack (the most recently opened, still-unclosed bracket); if not, invalid.

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'}
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: Unmatched closing bracket with an empty stack, e.g. ")(" — caught by 'not stack' check before popping.

Memory hook: "Push opens; a close must match the most recent open."

Design Min Stack

LeetCode #155 | **MEDIUM** | Pattern: Stack (Auxiliary Min Stack)

Video: "Design Min Stack - Amazon Interview Question - Leetcode 155 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Design a stack supporting push, pop, top, and getMin, all in $O(1)$ time.

Recognize it: "design a stack with $O(1)$ getMin/getMax" → parallel auxiliary min-stack.

BRUTE FORCE

Plain stack; getMin scans the whole stack every time it's called.

```
class MinStack:
    def __init__(self):
        self.stack = []
    def push(self, val):
        self.stack.append(val)
    def pop(self):
        self.stack.pop()
    def top(self):
        return self.stack[-1]
    def get_min(self):
        return min(self.stack)  #  $O(n)$  !
```

Complexity: push/pop/top $O(1)$, getMin $O(n)$ time | $O(n)$ space

OPTIMAL

Maintain a second stack that always stores the running minimum alongside each push, so it's popped in lockstep and always instantly available.

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []
    def push(self, val):
        self.stack.append(val)
        m = val if not self.min_stack else min(val, self.min_stack[-1])
        self.min_stack.append(m)
    def pop(self):
        self.stack.pop()
        self.min_stack.pop()
    def top(self):
        return self.stack[-1]
    def get_min(self):
        return self.min_stack[-1]
```

Complexity: $O(1)$ all ops time | $O(n)$ space

Edge case: Pushing duplicate minimum values then popping one — min_stack's per-push min() means the remaining copy is still correctly reported.

Memory hook: "A second stack, always remembering the min-so-far."

Daily Temperatures

LeetCode #739 | **MEDIUM** | Pattern: Monotonic Stack

Video: "Daily Temperatures - Monotonic Stack - Leetcode 739 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given daily temperatures, return an array where `answer[i]` is how many days until a warmer temperature; 0 if none exists.

Recognize it: "next greater/warmer element to the right" → Monotonic (decreasing) Stack.

BRUTE FORCE

For each day, scan forward until a warmer day is found.

```
def daily_temperatures(temps):
    n = len(temps)
    answer = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if temps[j] > temps[i]:
                answer[i] = j - i
                break
    return answer
```

Complexity: $O(n^2)$ time | $O(1)$ extra space

OPTIMAL

Keep a stack of day-indices whose warmer day hasn't been found yet. When a warmer day appears, it resolves every cooler day still waiting on the stack.

```
def daily_temperatures(temps):
    answer = [0] * len(temps)
    stack = [] # indices, decreasing temps
    for i, temp in enumerate(temps):
        while stack and temps[stack[-1]] < temp:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: Strictly decreasing temperatures — nothing ever gets popped, every answer stays 0.

Memory hook: "Stack of unresolved days; a warmer day resolves everyone smaller beneath it."

Evaluate Reverse Polish Notation

LeetCode #150 | **MEDIUM** | Pattern: Stack

Video: "Evaluate Reverse Polish Notation - Leetcode 150 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given tokens of a postfix arithmetic expression (+ - * /), evaluate it and return the integer result.

Recognize it: "postfix / Reverse Polish Notation evaluation" → Stack.

BRUTE FORCE

There's no simpler alternative to a stack here; a naive idea (convert to infix and use a general parser) is more complex, not simpler.

```
# A general infix-parser approach still needs precedence
# handling and is strictly more complex than the stack
# approach below for postfix notation - not shown as a
# separate 'slower but simpler' baseline; go straight to
# the stack-based evaluation, which is already optimal.
```

Complexity: N/A time | N/A space

OPTIMAL

Postfix means an operator always combines the two MOST RECENT operands — a stack naturally holds 'most recent' in order.

```
def eval_rpn(tokens):
    stack = []
    ops = {'+', '-', '*', '/'}
    for tok in tokens:
        if tok not in ops:
            stack.append(int(tok))
        else:
            b = stack.pop()
            a = stack.pop()
            if tok == '+': stack.append(a + b)
            elif tok == '-': stack.append(a - b)
            elif tok == '*': stack.append(a * b)
            else: stack.append(int(a / b)) # truncate toward 0
    return stack.pop()
```

Complexity: O(n) time | O(n) space

Edge case: Negative division, e.g. -7/2 — use int(a/b), not a//b, since // floors toward -inf but this problem truncates toward 0.

Memory hook: "Numbers pile up; an operator pops the top two and pushes the result."

Implement Queue using Stacks

LeetCode #232 | **EASY** | Pattern: Stack (Two-Stack Trick)

Video: "Implement Queue using Stacks - Leetcode 232 - Python" — NeetCode/O. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Implement a FIFO queue using only two stacks, supporting push, pop, peek, and empty.

Recognize it: "implement X using only Y" → look for a two-structure trick (here: two stacks, one for in, one for out).

BRUTE FORCE

Use one stack for input; to pop/peek (need the OLDEST item), pop everything into a temp list, take the item, then push everything back — done every single call.

```
class MyQueue:
    def __init__(self):
        self.stack = []
    def push(self, x):
        self.stack.append(x)
    def pop(self):
        temp = []
        while len(self.stack) > 1:
            temp.append(self.stack.pop())
        result = self.stack.pop()
        while temp:
            self.stack.append(temp.pop())
        return result  # O(n) every call!
```

Complexity: pop/peek $O(n)$ every call time | $O(n)$ space

OPTIMAL

Two stacks: 'in_stack' for pushes, 'out_stack' for pops. Only transfer in→out when out_stack is empty; each element is moved at most once, ever.

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []
    def push(self, x):
        self.in_stack.append(x)
    def _shift(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
    def pop(self):
        self._shift()
        return self.out_stack.pop()
    def peek(self):
        self._shift()
        return self.out_stack[-1]
    def empty(self):
        return not self.in_stack and not self.out_stack
```

Complexity: $O(1)$ amortized time | $O(n)$ space

Edge case: Interleaved push/pop calls — out_stack only refills once it's fully drained, so relative FIFO order is still preserved correctly.

Memory hook: "Two stacks: reverse once into 'out', drain it fully before refilling."

Implement Stack using Queues

LeetCode #225 | **EASY** | Pattern: Queue (Rotation Trick)

Video: "Implement Stack using Queues - Leetcode 225 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Implement a LIFO stack using only queues, supporting push, pop, top, and empty.

Recognize it: "implement X using only Y" → use a rotation trick so the newest element ends up at the front of the queue.

BRUTE FORCE

Use two queues; to push, add to the empty one, then move everything from the other queue behind it, then swap names — $O(n)$ per push done via an extra queue.

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()
    def push(self, x):
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1
    def pop(self):
        return self.q1.popleft()
    def top(self):
        return self.q1[0]
    def empty(self):
        return not self.q1
```

Complexity: push $O(n)$ time | $O(n)$ space

Edge case: Single-element stack — the rotation loop runs zero times ($\text{len}(q)-1 == 0$), element just stays at the front.

Memory hook: "Rotate the queue so the newest push lands at the front."

OPTIMAL

Single queue: after appending the new element, rotate the queue by rotating everything BEFORE it to behind it — the new element ends up at the front, acting like the top of a stack.

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())
    def pop(self):
        return self.q.popleft()
    def top(self):
        return self.q[0]
    def empty(self):
        return not self.q
```

Complexity: push $O(n)$, pop/top $O(1)$ time | $O(n)$ space

Maximum Depth of Binary Tree

LeetCode #104 | **EASY** | Pattern: DFS / Recursion

Video: "Maximum Depth of Binary Tree - 3 Solutions - Leetcode 104 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given the root of a binary tree, return its maximum depth (number of nodes on the longest root-to-leaf path).

Recognize it: "depth/height of a tree" → DFS Recursion: $1 + \max(\text{left}, \text{right})$.

BRUTE FORCE

There isn't a distinct slower approach here; the natural solution IS the recursive one below. An alternative worth showing is iterative BFS (level counting).

```
from collections import deque

def max_depth_bfs(root):
    if not root:
        return 0
    depth = 0
    queue = deque([root])
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
    return depth
```

Complexity: $O(n)$ time | $O(n)$ worst case space

Edge case: Empty tree (root=None) — base case returns 0 immediately.

Memory hook: "1 plus whichever child subtree goes deeper."

OPTIMAL

Depth of a tree = $1 + \max(\text{depth of left subtree}, \text{depth of right subtree})$; null node has depth 0. Direct recursive definition.

```
def max_depth(root):
    if root is None:
        return 0
    left = max_depth(root.left)
    right = max_depth(root.right)
    return 1 + max(left, right)
```

Complexity: $O(n)$ time | $O(h)$ space

Binary Tree Level Order Traversal

LeetCode #102 | **MEDIUM** | Pattern: BFS

Video: "Binary Tree Level Order Traversal - BFS - Leetcode 102" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given the root of a binary tree, return the node values grouped level by level (left to right), as a list of lists.

Recognize it: "process a tree/grid level by level" → BFS with a queue, snapshotting queue size per level.

BRUTE FORCE

DFS could work but requires tracking depth explicitly and appending to the right list — more error-prone than BFS's natural level grouping.

```
def level_order_dfs(root):
    result = []
    def dfs(node, depth):
        if not node:
            return
        if depth == len(result):
            result.append([])
        result[depth].append(node.val)
        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)
    dfs(root, 0)
    return result
```

Complexity: $O(n)$ time | $O(n)$ space

OPTIMAL

BFS naturally processes one level at a time. Snapshot the queue's current size before processing, so that batch is exactly one level's worth of nodes.

```
from collections import deque

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: Empty tree — guard clause returns [] immediately.

Memory hook: "Snapshot queue size = process exactly one level."

Binary Tree Vertical Order Traversal

LeetCode #314 | **MEDIUM** | Pattern: BFS + Hash Map (Column Index)

Video: "Binary Tree Vertical Order Traversal - Leetcode 314 - Python" — NeetCodeIO. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a binary tree, return the node values grouped by vertical column, ordered left to right; within a column, top to bottom, and left-to-right for ties at the same node depth.

Recognize it: "vertical columns of a tree" → track a column index while doing BFS, group by column in a hash map.

BRUTE FORCE

DFS while tracking (column, row) pairs, then sort all (column, row, value) triples at the end.

```
def vertical_order_dfs(root):
    triples = []
    def dfs(node, col, row):
        if not node:
            return
        triples.append((col, row, node.val))
        dfs(node.left, col - 1, row + 1)
        dfs(node.right, col + 1, row + 1)
    dfs(root, 0, 0)
    triples.sort() # by col, then row, then value
    result = {}
    for col, row, val in triples:
        result.setdefault(col, []).append(val)
    return [result[c] for c in sorted(result)]
```

Complexity: $O(n \log n)$ time | $O(n)$ space

OPTIMAL

BFS processes nodes level by level (top to bottom, left to right) automatically, so within a column, values are already appended in correct order — no final sort needed, just group by column index and sort column KEYS at the end.

```
from collections import deque, defaultdict

def vertical_order(root):
    if not root:
        return []
    columns = defaultdict(list)
    queue = deque([(root, 0)])
    while queue:
        node, col = queue.popleft()
        columns[col].append(node.val)
        if node.left: queue.append((node.left, col - 1))
        if node.right: queue.append((node.right, col + 1))
    return [columns[c] for c in sorted(columns)]
```

Complexity: $O(n \log n)$ time | $O(n)$ space

Edge case: Two nodes landing in the same column AND same BFS level (siblings from different parents) — BFS's left-to-right level processing naturally orders them correctly.

Memory hook: "BFS order is already top-to-bottom, left-to-right — just bucket by column."

Clone Graph

LeetCode #133 | **MEDIUM** | Pattern: DFS/BFS + Hash Map

Video: "Clone Graph - Depth First Search - Leetcode 133" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a reference node in a connected undirected graph, return a deep copy (clone) of the entire graph.

Recognize it: "deep copy a graph/structure with arbitrary cross-references" → Hash Map from original node to its clone.

BRUTE FORCE

There isn't a meaningfully 'slower' correct alternative — you fundamentally need some way to avoid re-cloning (and infinitely looping on) a node you've already visited. Naive recursion WITHOUT a visited map would infinite-loop on any cycle.

```
# Recursion without tracking visited nodes:
def clone_graph_broken(node):
    if not node:
        return None
    copy = Node(node.val)
    for neighbor in node.neighbors:
        copy.neighbors.append(clone_graph_broken(neighbor))
    return copy
# BROKEN: infinite recursion on any cycle back to a
# node already being cloned (graphs here are typically cycl
```

Complexity: Infinite loop on cycles time | N/A space

OPTIMAL

A hash map from original node to its clone both (a) prevents infinite recursion on cycles and (b) lets neighbors correctly reference already-created clones.

```
def clone_graph(node):
    if not node:
        return None
    old_to_new = {}
    def dfs(n):
        if n in old_to_new:
            return old_to_new[n]
        copy = Node(n.val)
        old_to_new[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy
    return dfs(node)
```

Complexity: $O(V + E)$ time | $O(V)$ space

Edge case: Single node with a self-loop neighbor — map entry is created BEFORE recursing into neighbors, so the self-reference resolves to the already-created clone instead of looping forever.

Memory hook: "Map original → clone FIRST, then recurse into neighbors."

Course Schedule

LeetCode #207 | **MEDIUM** | Pattern: Graph DFS / Topological Sort (Cycle Detection)

Video: "Course Schedule - Graph Adjacency List - Leetcode 207" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given numCourses and prerequisite pairs [a, b] (must take b before a), return whether it's possible to finish all courses (i.e., the prerequisite graph has no cycle).

Recognize it: "can all tasks be completed given dependencies" → cycle detection in a directed graph (DFS with a recursion-stack marker, or Topological Sort / Kahn's algorithm).

BRUTE FORCE

Try every possible course order (permutations) and check if any order satisfies all prerequisites — combinatorially expensive.

```
from itertools import permutations

def can_finish_brute(numCourses, prerequisites):
    for order in permutations(range(numCourses)):
        position = {c: i for i, c in enumerate(order)}
        if all(position[b] < position[a] for a, b in prerequisites):
            return True
    return False # O(n!) - only feasible for tiny n
```

Complexity: O(n!) time | O(n) space

OPTIMAL

A valid course order exists iff the prerequisite graph has no cycle. DFS each course, marking nodes 'in current path'; if DFS revisits a node still in the current path, that's a cycle.

```
def can_finish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    for a, b in prerequisites:
        graph[a].append(b)
    state = [0] * numCourses # 0=unvisited, 1=visiting, 2=done
    def dfs(course):
        if state[course] == 1:
            return False # cycle!
        if state[course] == 2:
            return True
        state[course] = 1
        for pre in graph[course]:
            if not dfs(pre):
                return False
        state[course] = 2
        return True
    return all(dfs(c) for c in range(numCourses))
```

Complexity: O(V + E) time | O(V + E) space

Edge case: A course that depends on itself, e.g. [0,0] — DFS immediately revisits course 0 while it's still marked 'visiting' (state 1), correctly detecting the cycle.

Memory hook: "Mark 'in-progress'; revisiting an in-progress node means a cycle."

Binary Search

LeetCode #704 | **EASY** | Pattern: Binary Search

Video: "Binary Search - Leetcode 704 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a sorted array of integers and a target, return its index, or -1 if not present, in $O(\log n)$.

Recognize it: "sorted array + search for a value" → Binary Search.

BRUTE FORCE

Scan every element left to right until the target is found.

```
def search(nums, target):
    for i, num in enumerate(nums):
        if num == target:
            return i
    return -1
```

Complexity: $O(n)$ time | $O(1)$ space

OPTIMAL

Sorted order means comparing the middle element tells you which entire half to discard — halving the search space every step.

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Complexity: $O(\log n)$ time | $O(1)$ space

Edge case: Target not present, e.g. searching for a value between two elements — left ends up greater than right, loop exits, returns -1.

Memory hook: "Middle tells you which whole half to throw away."

Find Minimum in Rotated Sorted Array

LeetCode #153 | **MEDIUM** | Pattern: Binary Search (Modified)

Video: "Find Minimum in Rotated Sorted Array - Binary Search - Leetcode 153 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given a rotated sorted array with all-unique values, find the minimum element in $O(\log n)$.

Recognize it: "rotated sorted array" → Binary Search comparing mid against the right boundary to decide which half is 'still sorted'.

BRUTE FORCE

Scan the whole array to find the minimum directly.

```
def find_min(nums):  
    return min(nums) # O(n), ignores the sorted structure
```

Complexity: $O(n)$ time | $O(1)$ space

OPTIMAL

Compare $\text{nums}[\text{mid}]$ to $\text{nums}[\text{right}]$. If $\text{nums}[\text{mid}] > \text{nums}[\text{right}]$, the minimum is in the right half (rotation point is there); otherwise it's in the left half (mid could BE the minimum, so don't exclude it).

```
def find_min(nums):  
    left, right = 0, len(nums) - 1  
    while left < right:  
        mid = (left + right) // 2  
        if nums[mid] > nums[right]:  
            left = mid + 1  
        else:  
            right = mid  
    return nums[left]
```

Complexity: $O(\log n)$ time | $O(1)$ space

Edge case: Array not rotated at all (already fully sorted) — $\text{nums}[\text{mid}]$ is never greater than $\text{nums}[\text{right}]$, so right keeps shrinking to index 0, correctly returning $\text{nums}[0]$.

Memory hook: "Compare mid to the right edge — that tells you which side hides the rotation."

LRU Cache

LeetCode #146 | **MEDIUM** | Pattern: Hash Map + Doubly Linked List

Video: "LRU Cache - Twitch Interview Question - Leetcode 146" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Design a Least Recently Used (LRU) cache with a fixed capacity, supporting `get(key)` and `put(key, value)`, both in $O(1)$ average time.

Recognize it: "design a cache with $O(1)$ get/put and eviction of the least recently used item" → Hash Map ($O(1)$ lookup) + Doubly Linked List ($O(1)$ reorder/evict).

BRUTE FORCE

Use a plain dict for values plus a separate list tracking usage order; every get/put requires an $O(n)$ scan or shift of that order list.

```
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.data = {}
        self.order = [] # least-recent at index 0
    def get(self, key):
        if key not in self.data:
            return -1
        self.order.remove(key) #  $O(n)$ 
        self.order.append(key)
        return self.data[key]
    def put(self, key, value):
        if key in self.data:
            self.order.remove(key) #  $O(n)$ 
        elif len(self.data) >= self.cap:
            oldest = self.order.pop(0) #  $O(n)$ 
            del self.data[oldest]
        self.data[key] = value
        self.order.append(key)
```

Complexity: $O(n)$ per get/put time | $O(n)$ space

OPTIMAL

A hash map gives $O(1)$ lookup by key; a doubly linked list gives $O(1)$ removal/insertion at any position (no shifting) to track recency order, moving a node to the front on every access.

```
class Node:
    def __init__(self, k, v):
        self.key, self.val = k, v
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.left = Node(0, 0) # least recent side
        self.right = Node(0, 0) # most recent side
        self.left.next = self.right
        self.right.prev = self.left
    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
    def _insert(self, node):
        node.prev, node.next = self.right.prev, self.right
        self.right.prev.next = node
        self.right.prev = node
    def get(self, key):
        if key not in self.map:
            return -1
        self._remove(self.map[key]); self._insert(self.map[key])
        return self.map[key].val
    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])
        node = Node(key, value)
        self.map[key] = node
        self._insert(node)
        if len(self.map) > self.cap:
            lru = self.left.next
            self._remove(lru); del self.map[lru.key]
```

Complexity: $O(1)$ get/put time | $O(\text{capacity})$ space

Edge case: Updating an existing key's value — must remove the old node and reinsert (or move) it to the most-recent end, not just overwrite val in place.

Memory hook: "Map for $O(1)$ lookup, doubly linked list for $O(1)$ reorder — evict from the far end."

Random Pick with Weight

LeetCode #528 | **MEDIUM** | Pattern: Prefix Sum + Binary Search

Video: "Random Pick with Weight | Bucket approach | Leetcode #528 | Binary search" — Techdose. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an array of positive weights, design a structure that picks an index at random, where the probability of picking index i is proportional to $\text{weights}[i]$.

Recognize it: "pick randomly with weighted probability" → Prefix Sum turns weights into ranges; Binary Search finds which range a random point falls into.

BRUTE FORCE

Build a 'bucket' array where each index appears $\text{weights}[i]$ times, then pick a uniformly random element from that array directly.

```
import random

class Solution:
    def __init__(self, w):
        self.buckets = []
        for i, weight in enumerate(w):
            self.buckets.extend([i] * weight) # O(total weight)
    def pick_index(self):
        return random.choice(self.buckets)
```

Complexity: build $O(\sum(w))$, pick $O(1)$ time | $O(\sum(w))$ space

OPTIMAL

Build a prefix-sum array of weights (cumulative ranges). Pick a random number in $[1, \text{total}]$, then binary search for the first prefix sum $\geq$ that number — that index's range contains the random point.

```
import random, bisect

class Solution:
    def __init__(self, w):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
        self.total = total
    def pick_index(self):
        target = random.randint(1, self.total)
        return bisect.bisect_left(self.prefix, target)
```

Complexity: build $O(n)$, pick $O(\log n)$ time | $O(n)$ space

Edge case: A single weight of value 1 — $\text{prefix}=[1]$, any random target (must be 1) maps to index 0 correctly.

Memory hook: "Weights become ranges (prefix sums); binary search finds which range the dart landed in."

Rotting Oranges

LeetCode #994 | **MEDIUM** | Pattern: Multi-Source BFS

Video: "Rotting Oranges - Leetcode 994 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: A grid has empty cells, fresh oranges, and rotten oranges. Every minute, rot spreads to adjacent fresh oranges. Return the minutes until no fresh orange remains, or -1 if impossible.

Recognize it: "spreads simultaneously from multiple sources over discrete time steps" → Multi-Source BFS (each BFS level = one minute).

BRUTE FORCE

Simulate minute by minute: rescan the WHOLE grid every minute to find newly-infected oranges.

```
def oranges_rotting_brute(grid):
    rows, cols = len(grid), len(grid[0])
    minutes = 0
    while True:
        fresh_left = False
        to_rot = []
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    fresh_left = True
                    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                        nr, nc = r+dr, c+dc
                        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                            to_rot.append((r,c))
                            break
        if not to_rot:
            return -1 if fresh_left else minutes
        for r, c in to_rot:
            grid[r][c] = 2
        minutes += 1
```

Complexity: $O((rows*cols)^2)$ time | $O(rows*cols)$ space

OPTIMAL

Seed a BFS queue with EVERY rotten orange at once. Each full BFS level = exactly one minute of simultaneous spreading; no rescanning needed.

```
from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: queue.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    dirs = [(-1,0),(1,0),(0,-1),(0,1)]
    while queue and fresh > 0:
        minutes += 1
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for dr, dc in dirs:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
    return minutes if fresh == 0 else -1
```

Complexity: $O(rows * cols)$ time | $O(rows * cols)$ space

Edge case: A fresh orange fully sealed off by walls/empty cells with no path to any rotten orange — fresh never reaches 0, correctly returns -1.

Memory hook: "Seed BFS with every rotten orange at once — BFS levels ARE the minutes."

Maximum Subarray

LeetCode #53 | **MEDIUM** | Pattern: Dynamic Programming (Kadane's Algorithm)

Video: "Maximum Subarray - Amazon Coding Interview Question - Leetcode 53 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an integer array (possibly with negatives), find the contiguous subarray with the largest sum and return that sum.

Recognize it: "contiguous subarray, maximum sum" → Kadane's Algorithm (running sum, reset when it goes negative).

BRUTE FORCE

Check every possible subarray's sum directly (or extend a running sum per start index).

```
def max_subarray(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

Complexity: $O(n^2)$ time | $O(1)$ space

OPTIMAL

If the running sum ever goes negative, it can only hurt any future subarray — drop it and restart at the current element instead of carrying it forward.

```
def max_subarray(nums):
    current = best = nums[0]
    for num in nums[1:]:
        current = max(num, current + num)
        best = max(best, current)
    return best
```

Complexity: $O(n)$ time | $O(1)$ space

Edge case: All-negative array, e.g. $[-3, -1, -2]$ — current/best start at $\text{nums}[0]$ (not 0), so the answer correctly becomes the least-negative single element, -1.

Memory hook: "A negative running sum is dead weight — drop it and start fresh."

Top K Frequent Elements

LeetCode #347 | **MEDIUM** | Pattern: Bucket Sort (or Heap)

Video: "Top K Frequent Elements - Bucket Sort - Leetcode 347 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

Problem: Given an integer array and an integer k, return the k most frequently occurring elements, in any order.

Recognize it: "top k frequent / most common" → count frequencies, then Bucket Sort by frequency for true $O(n)$ (or a size-k heap for $O(n \log k)$).

BRUTE FORCE

Count frequencies, then fully sort all distinct values by count and take the top k.

```
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    sorted_items = sorted(counts.items(), key=lambda x: -x[1])
    return [val for val, cnt in sorted_items[:k]]
```

Complexity: $O(n \log n)$ time | $O(n)$ space

OPTIMAL

Frequency can never exceed n, so make 'buckets' indexed by frequency ($0..n$); put each value in its frequency's bucket, then read off values from the highest bucket down until k are collected — no comparison sort needed.

```
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    n = len(nums)
    buckets = [[] for _ in range(n + 1)]
    for val, freq in counts.items():
        buckets[freq].append(val)
    result = []
    for freq in range(n, 0, -1):
        for val in buckets[freq]:
            result.append(val)
            if len(result) == k:
                return result
    return result
```

Complexity: $O(n)$ time | $O(n)$ space

Edge case: k equals the number of distinct elements — loop collects everything before returning, naturally covering that case.

Memory hook: "Bucket index = frequency — no sorting needed, frequency is bounded by n."

Pattern Cheat Sheet

Interview clue	Pattern
Need a pair/complement fast	Hash Map (Two Sum, Contains Duplicate)
Same letters/counts grouped	Hash Map canonical key (Group Anagrams)
Product/sum except self, no division	Prefix * Suffix products
# subarrays summing to k, negatives allowed	Prefix Sum + Hash Map
Palindrome check, O(1) space	Two Pointers from both ends
Maximize area/container between two points	Two Pointers, move the shorter side
Longest substring with a constraint	Sliding Window
Matching brackets / most-recent-first	Stack
Next greater/warmer element to the right	Monotonic Stack
Design O(1) getMin alongside a stack	Auxiliary min-stack
Implement stack with queues (or vice versa)	Two-structure rotation trick
Tree depth / recursive tree property	DFS Recursion
Process a tree/grid level by level	BFS with queue-size snapshot
Deep copy a graph/structure with cycles	DFS/BFS + Hash Map (orig -> clone)
Can all tasks finish given dependencies	Cycle detection / Topological Sort
Sorted array + search	Binary Search
Rotated sorted array	Binary Search vs. right boundary
O(1) get/put + evict least-recently-used	Hash Map + Doubly Linked List
Pick randomly with weighted probability	Prefix Sum + Binary Search
Multiple sources spreading simultaneously	Multi-Source BFS
Max sum of a contiguous subarray	Kadane's Algorithm (DP)
Top K frequent, bound on frequency	Bucket Sort by frequency

Sources

Primary source

The playlist at youtube.com/playlist?list=PL_6hP3Z1GQqYrrF1fETaEpkLAX6M8E7j was provided. This session's network policy blocks youtube.com, so none of its videos could be fetched or transcribed. No quotes in this guide are attributed to, or copied from, any video transcript.

Supporting sources

LeetCode problem statements (for exact constraints/return formats) and general, widely-known algorithmic technique names and explanations, written independently.

Videos named in the provided playlist (titles/channels, not transcribed)

- "Two Sum - Leetcode 1 - HashMap - Python" — NeetCode
- "Contains Duplicate - Leetcode 217 - Python" — NeetCode
- "Group Anagrams - Categorize Strings by Count - Leetcode 49" — NeetCode
- "Product of Array Except Self - Leetcode 238 - Python" — NeetCode
- "Subarray Sum Equals K - Prefix Sums - Leetcode 560 - Python" — NeetCode
- "Valid Palindrome - Leetcode 125 - Python" — NeetCode
- "Container with Most Water - Leetcode 11 - Python" — NeetCode
- "Longest Substring Without Repeating Characters - Leetcode 3 - Python" — NeetCode
- "Minimum Window Substring - Airbnb Interview Question - Leetcode 76" — NeetCode
- "Valid Parentheses - Stack - Leetcode 20 - Python" — NeetCode
- "Design Min Stack - Amazon Interview Question - Leetcode 155 - Python" — NeetCode
- "Daily Temperatures - Monotonic Stack - Leetcode 739 - Python" — NeetCode
- "Evaluate Reverse Polish Notation - Leetcode 150 - Python" — NeetCode
- "Implement Queue using Stacks - Leetcode 232 - Python" — NeetCodeIO
- "Implement Stack using Queues - Leetcode 225 - Python" — NeetCode
- "Maximum Depth of Binary Tree - 3 Solutions - Leetcode 104 - Python" — NeetCode
- "Binary Tree Level Order Traversal - BFS - Leetcode 102" — NeetCode
- "Binary Tree Vertical Order Traversal - Leetcode 314 - Python" — NeetCodeIO
- "Clone Graph - Depth First Search - Leetcode 133" — NeetCode
- "Course Schedule - Graph Adjacency List - Leetcode 207" — NeetCode
- "Binary Search - Leetcode 704 - Python" — NeetCode
- "Find Minimum in Rotated Sorted Array - Binary Search - Leetcode 153 - Python" — NeetCode
- "LRU Cache - Twitch Interview Question - Leetcode 146" — NeetCode
- "Random Pick with Weight | Bucket approach | Leetcode #528 | Binary search" — Techdose
- "Rotting Oranges - Leetcode 994 - Python" — NeetCode
- "Maximum Subarray - Amazon Coding Interview Question - Leetcode 53 - Python" — NeetCode
- "Top K Frequent Elements - Bucket Sort - Leetcode 347 - Python" — NeetCode